API Security Risk Analysis

Prepared by: IMAN BEDRU

Program: Cybersecurity Internship / Training Task

Date: February 2026

Table of Contents

1.Introduction.....5
2.Objective.....5
3.Tools And Methdology.....6
4.DummyJSON API Endpoints Tested.....7
5.Findings and Observations.....8
6.Security Risk Analysis.....13
8.Recommendations.....16
9.Conclusion.....18
10.Appendices.....18.

1 .Introduction

This report presents the security analysis of the DummyJSON API (https://dummyjson.com/). The goal was to test the API endpoints for functionality, inspect responses and headers, and assess potential security risks.

2.Objective

- Test multiple endpoints of the DummyJSON API.
- Analyze responses, status codes, and headers.
- Identify potential security weaknesses and provide recommendations.

3. Tools and Methodology

Tools Used:

- **Insomnia** – For sending GET/POST requests to DummyJSON API.
- **Browser Developer Tools** – To inspect headers and responses.
- **Postman**-Api testing

Methodology:

1. Identify key endpoints from DummyJSON documentation.
2. Send requests with valid and invalid inputs.
3. Record response status codes, headers, and JSON content.
4. Evaluate security implications and suggest improvements.

4. DummyJSON API Endpoints Tested

#	Endpoint URL	Method	Status Code	Response Example	Notes
1	/products	GET	200	List of products (JSON array)	Public endpoint
2	/products/99999	GET	404	{"message":"Product not found"}	Nonexistent product test
3	/users	GET	200	List of users (JSON array)	No auth required
4	/users/150	GET	404	{"message":"User not found"}	Invalid user ID
5	/auth/login	POST	200 / 400	Token / error	Validates input
6	/carts	GET	200	List of carts (JSON array)	Public endpoint

My Collection > **Get data**

Save | Share

GET https://dummyjson.com/products/99999 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 404 Not Found • 572 ms • 967 B | Save Response

JSON | Preview | Debug with AI

```
1 {
2     "message": "Product with id '99999' not found"
3 }
```

My Collection > **Get data**

Save | Share

GET https://dummyjson.com/products/1 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 488 ms • 2.66 KB | Save Response

JSON | Preview | Visualize


```
1 {
2     "id": 1,
3     "title": "Essence Mascara Lash Princess",
4     "description": "The Essence Mascara Lash Princess is a popular mascara known for
          its volumizing and lengthening effects. Achieve dramatic lashes with this
          long-lasting and cruelty-free formula.",
5     "category": "beauty",
6     "price": 9.99,
7     "discountPercentage": 10.48,
8     "rating": 2.56,
9     "stock": 99,
10    "tags": [
```

My Collection > **Get data**

Save | Share

GET https://dummyjson.com/products Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK • 1.78 s • 44.02 KB | Save Response

JSON | Preview | Visualize


```
{
    "products": [
        {
            "id": 1,
            "title": "Essence Mascara Lash Princess",
            "description": "The Essence Mascara Lash Princess is a popular mascara
                known for its volumizing and lengthening effects. Achieve dramatic
                lashes with this long-lasting and cruelty-free formula.",
            "category": "beauty",
            "price": 9.99,
            "discountPercentage": 10.48,
            "rating": 2.56
```


https://dummyjson.com/users/1

Save | Share

GET https://dummyjson.com/users/1 Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookie

Body | 200 OK • 494 ms • 2.31 KB

JSON | Preview | Visualize


```
    "macAddress": "47:fa:41:18:ec:eb",
    "university": "University of Wisconsin--Madison",
    "bank": {
        "cardExpire": "05/28",
        "cardNumber": "3693233511855044",
        "cardType": "Diners Club International",
        "currency": "GBP",
        "iban": "GB74MH2UZLR9TRPHYNU8F8"
    },
    "company": {
        "department": "Engineering",
        "name": "Dooley, Kozey and Cronin",
        "title": "Sales Manager",
        "address": {
            "address": "263 Tenth Street",
```

https://dummyjson.com/users

Save | Share

GET https://dummyjson.com/users Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Body | 200 OK • 3.22 s • 41.88 KB

JSON | Preview | Visualize


```
24            "ip": "42.48.100.32",
25            "address": {
26                "address": "626 Main Street",
27                "city": "Phoenix",
28                "state": "Mississippi",
29                "stateCode": "MS",
30                "postalCode": "29112",
31                "coordinates": {
32                    "lat": -77.16213,
33                    "lng": -92.084824
34                },
35                "country": "United States"
36            },
37            "macAddress": "47:fa:41:18:ec:eb",
38            "university": "University of Wisconsin--Madison",
```


https://dummyjson.com/users

Save | Share

GET https://dummyjson.com/users Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Body | 200 OK • 3.22 s • 41.88 KB

JSON | Preview | Visualize


```
1  {
2      "users": [
3          {
4              "id": 1,
5              "firstName": "Emily",
6              "lastName": "Johnson",
7              "maidenName": "Smith",
8              "age": 29,
9              "gender": "female",
10             "email": "emily.johnson@x.dummyjson.com",
11             "phone": "+81 965-431-3024",
12             "username": "emilys",
13             "password": "emilyspass",
14             "birthDate": "1996-5-30",
15             "image": "https://dummyjson.com/icon/emilys/128",
```

5. Findings and Observations

- 404 Responses: Correctly returned for invalid product/user IDs (/products/99999, /users/150).
- Public access: Many endpoints like /products and /users are publicly accessible without authentication.
- Response headers:
- CORS is enabled for cross-origin requests.
- No sensitive information exposed in headers.
- Server info is not disclosed.
- Input validation: /auth/login returns proper error messages for invalid inputs.

6. Security Risk Analysis

Risk ID	Risk Description	Severity	Endpoint Affected	Recommendation
1	Public access to all user and product data	Medium	/products, /users	Require auth for sensitive endpoints
2	Lack of rate limiting	Medium	/auth/login	Implement throttling/rate limiting
3	Missing HSTS headers	Low	All	Enable HSTS to enforce HTTPS

- Overall, DummyJSON is a dummy/test API, so the security risks are mostly theoretical, but this exercise demonstrates proper API testing methodology.

7. Recommendations

Based on the observations made during the analysis of the DummyJSON API, several improvements can be suggested to enhance API security and align with industry best practices.

1. Implement Authentication and Authorization

- Currently, many endpoints appear publicly accessible. In real-world applications, sensitive resources such as user data should require authentication. Implementing secure authentication mechanisms such as JWT (JSON Web Tokens) or OAuth would ensure that only authorized users can access protected resources.

2. Introduce Rate Limiting

- To prevent abuse or brute-force attacks, rate limiting should be applied to critical endpoints such as login or data retrieval endpoints. Limiting the number of requests per user or IP address can reduce the risk of automated attacks and denial-of-service attempts.

3. Improve Security Headers

Security headers help protect APIs and web applications from common attacks. It is recommended to implement headers such as:

- Strict-Transport-Security (HSTS) to enforce HTTPS connections
- Content-Security-Policy (CSP) to mitigate cross-site scripting attacks
- X-Content-Type-Options to prevent MIME-type sniffing
- These headers help improve the overall security posture of the API.

4. Strengthen Input Validation

- All user inputs should be validated and sanitized to prevent injection attacks such as SQL injection or command injection. Proper validation ensures that only expected data formats and values are accepted by the API.

5. Implement Logging and Monitoring

- Logging API requests and monitoring unusual activity can help detect suspicious behavior early. Monitoring tools can alert administrators about potential attacks, repeated failed login attempts, or abnormal traffic patterns.

6. Hide Server and Technology Information

- API responses and headers should avoid revealing server versions or technology stack information. Such details can help attackers identify vulnerabilities associated with specific software versions.

7. Apply Principle of Least Privilege

- Endpoints should only expose the minimum amount of information necessary for functionality. Restricting unnecessary data exposure helps reduce the risk of information leakage.

8. Conclusion

The testing of the DummyJSON API demonstrated how API endpoints respond to valid and invalid requests. The analysis showed proper handling of non-existent resources through 404 responses and consistent JSON outputs.

Although the API exposes many endpoints publicly, this behavior is expected because the platform is designed for testing and learning purposes rather than production use.

Through this task, important skills were practiced, including:

- API endpoint testing
- HTTP status code analysis
- Header inspection
- Basic security risk evaluation

These techniques are essential for identifying potential vulnerabilities in real-world APIs.

9. Disclaimer

This report analyzes the DummyJSON API solely for educational and training purposes as part of the Future Interns API Security Risk Analysis task.

The DummyJSON service is a publicly available mock API designed for development and testing, and therefore some security practices (such as authentication or restricted access) may not be fully implemented because the API is not intended for production use.

No malicious activity, exploitation, or unauthorized access attempts were performed during this analysis. All testing was conducted using standard API requests to observe responses and understand API behavior.